COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	06/08/2026	**DATE SUBMITTED:**	08/08/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 5.1 **Queues**

1. PROCEDURE OUTPUT

[ILOA]

[SOURCE CODE]

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> q) {
    std::queue <char> c = q;
    while (!c.empty()) {
        std::cout << c.front() << " ";
        c.pop();
    }
    std::cout << std::endl;
}

int main() {
    std::queue<char> a;
    a.push('J');
    a.push('A');
    a.push('M');
    a.push('E');
    a.push('S');

    std::cout << "Queue a is: ";
    display(a);

    std::cout << "a.empty() : " << a.empty() << std::endl;
    std::cout << "a.size() : " << a.size() << std::endl;
    std::cout << "a.front() : " << a.front() << std::endl;
    std::cout << "a.back() : " << a.back() << std::endl;

    std::cout << "a.pop() : ";
    a.pop();
    display(a);
```

```
32
33      a.push('J');
34      std::cout << "The queue a is : ";
35      display(a);
36
37      return 0;
38
39  }
```

[OUTPUT]

```
Queue a is: J A M E S
a.empty() : 0
a.size() : 5
a.front() : J
a.back() : S
a.pop() : A M E S
The queue a is : A M E S J
```

[ANALYSIS]

In the ILOA, This code source program that we were tasked to create demonstrates the **core principle** of **First-In, First Out (FIFO)** Queue using the **C++ Standard Template Library (STL).** By initializing a **std::queue** specialized for the **<char>** data type, it represents how to code showcases the elements in a sequential append to the back of the line and are initialized / processed first from the front. This code implementation consists of many inside functions from the library, .**push()** method operations are executed to insert an element to the container. The first character inserted **'J'** automatically claims the **front** or in terms of linked list, the **head position**. While the final character, as the function a.**back()** initializes, is the **'S'** character that is in the **rear** or tail position. This helps in initial data values in memory and provides a clean and organized dataset ready for **structural iteration and removal**.

[ILOB - LINKED LIST]

[SOURCE CODE]

```
#include <iostream>
#include "queueLL.h"
#include "queueArr.h"
#include <queue>
```

```cpp
int main() {
    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    std::cout << "enqueueing..." << std::endl;

    enqueue(1, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl; // Output: 1
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl; // Output: 1

    display(front);

    std::cout << "dequeueing..." << std::endl;

    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl; // Output: 2
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << std::endl; // Output: 3
    dequeue(&front, &back);
    dequeue(&front, &back); // Output: Queue is empty

    display(front);
```

```cpp
#ifndef QUELL_H
#define QUELL_H
#include <iostream>

template <typename T>
class qNode {
    public:
        T data;
        qNode<T>* next;
};
```

```
template <typename T>
qNode<T> *new_Node(T newData) {
    qNode<T> *newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = NULL;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>**backPtr) {
    //create a new node:
    qNode<T> *newNode = new_Node(newData);


    // how do we know that the queue is empty? if the front and back pointers are both null, then the queue is empty
    if(*frontPtr == nullptr && (*backPtr) == nullptr){
        *frontPtr = newNode;
        *backPtr = newNode;
        return;
    }

    //point the back next to the newNode:
    (*backPtr)->next = newNode;
    //the newnode will be the new backPtr:
    (*backPtr) = newNode;
    return;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    //check if the queue is empty:
    if(*frontPtr == nullptr && *backPtr == nullptr){
        std::cout << "Queue is empty" << std::endl;
```

```
        return;
    }

    //store the front node in a temporary variable:
    qNode<T>* tempPtr = (*frontPtr);

    if ((*frontPtr) == (*backPtr)) {
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete tempPtr;
        return;
    }

    //move the front pointer to the next node:
    (*frontPtr) = (*frontPtr)->next;

    tempPtr->next = nullptr;

    //delete the temporary variable:
    delete tempPtr;
}

template <typename T>
void display(qNode<T>* frontPtr) {
    if (frontPtr == nullptr) {
        std::cout << "Queue is empty" << std::endl;
        return;
    }

    qNode<T>* current = frontPtr;
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
```

```
}

#endif
```

[OUTPUT]

```
enqueueing...
front: 1 back: 1
front: 1 back: 3
1 2 3
dequeueing...
front: 2 back: 3
front: 3 back: 3
Queue is empty
Queue is empty
```

[ANALYSIS]

In the linked list implementation of the Queue, It is easier to understand the concept form of transitioning **between nodes**. The linked list approach is always **recycled through the approach of queue**. Indicating the **qNode class structure**, it represents the **basic building block** for our **data collection** as it is the one that creates a template for another node that can hold any data type. Inside it defines 2 variables, named **data** which **holds** the **actual value** and a **pointer** named **next** that **links a node to the next node**.

The new_Node() function is the one that creates new nodes in memory dynamically. It is the function that **saves the incoming data inside the node** and **sets its next pointer to NULL** so it starts completely clean indicated as:

```
newNode->data = newData;
newNode->next = NULL;
```

The **enqueue() function** is the line code that handles adding a new node to the very back of the queue. It uses double pointers so it can directly modify the original front and back tracking location. First inside the function is that we have a **condition** where **if the queue is empty**, the **function sets both the front and back to point** to the **new node**. **IF there is already data in** line, it **links** the **old back data's next pointer** to the **new node**. indicated as:

```
// how do we know that the queue is empty? if the front and
if(*frontPtr == nullptr && (*backPtr) == nullptr){
    *frontPtr = newNode;
    *backPtr = newNode;
    return;
}
```

```
//point the back next to the newNode:
(*backPtr)->next = newNode;
//the newnode will be the new backPtr:
(*backPtr) = newNode;
return;
```

The **dequeue() function** on another part, is the line of code that **removes data / nodes** from the front of the line and deletes them to **keep the system memory clean**. As its **conditions**, it **first checks if the queue is empty**, and it **prints a message stopping if there is nothing to remove**. BUT **if there is still data that exists**, it **creates a temporary pointer to hold the current front node** while it updates the iteration. If there is only one node left, it resets both boundaries to **null.** OTHERWISE, it keeps going forward to the next node in line before deleting the old front node.

```
if(*frontPtr == nullptr && *backPtr == nullptr){
    std::cout << "Queue is empty" << std::endl;
    return;
}
```

And lastly, the **display function()**, it is the line of code that walks through and prints the entire queue from start to finish. Inside its functions, has a condition that checks if the front pointer is empty, it stops if there's nothing to print. If the queue has data, it **creates a temporary node pointer** that starts at the front and uses a **loop function to print out each of the data one by one**. Once the temporary pointer hits the end of the chain or already **points to nullptr**, it stops the iteration.

```
qNode<T>* current = frontPtr;
while (current != nullptr) {
    std::cout << current->data << " ";
    current = current->next;
}
```

[ILOB - ARRAY]

[SOURCE CODE]

```
queueArr<int> q(6);

q.Enqueue(1);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
q.Enqueue(2);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
q.Enqueue(3);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
q.Enqueue(4);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
q.Enqueue(5);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
q.Enqueue(6);
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
q.Enqueue(7);
q.Dequeue();
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
q.Dequeue();
q.Dequeue();
q.Dequeue();
q.Dequeue();
q.Dequeue();
std::cout << "Front: " << q.Front() << " Back: " << q.Back() << " Size: " << q.Size() << std::endl;
```

[HEADER FILE]

```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>
```

```
template <typename T>
class queueArr {
   private:
      //pointer to dynamically allocated array
      T*  q_array;
      size_t q_capacity;
      size_t q_size;
      int q_front, q_back;

   public:
      //constructor
      queueArr(size_t q_capacity);

      bool Empty();
      size_t Size();
      void Clear();

      T Front();
      T Back();

      void Enqueue(T value);
      void Dequeue();

      //copy constructor:
      queueArr(const queueArr& other);

      //copy assignment operator
      queueArr& operator = (const queueArr& other);

      //destructor
      ~queueArr();
};
```

```
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}

template <typename T>
bool queueArr<T>::Empty(){
    return q_size == 0;
}

template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

template <typename T>
void queueArr<T>::Clear(){
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

template <typename T>
T queueArr<T>::Front(){

    if(Empty()){
        std::cout << "The Queue is Empty.\n";
        return T{};
    }

    return q_array[q_front];
```

```
}

template <typename T>
T queueArr<T>::Back(){

    if(Empty()){
        std::cout << "The Queue is Empty.\n";
        return T{};
    }

    return q_array[q_back];
}

template <typename T>
void queueArr<T>::Enqueue(T value){
    //check if the queue is full
    if(q_size == q_capacity){
        std::cout << "The Queue is Full.\n";
        return;
    }
    //compute for the q_back for circular (use the modulo operator)
    q_back = (q_back + 1) % q_capacity;
    // assign the q_array with the updated q_back to the value
    q_array[q_back] = value;
    //increment the q_size
    q_size++;
}

template <typename T>
void queueArr<T>::Dequeue(){
    //check if the queue is empty
    if(Empty()){
        std::cout << "The Queue is Empty.\n";
        return;
    }
```

```
    //compute for the q_front for circular (use the modulo operator)
    q_front = (q_front + 1) % q_capacity;
    //decrement the q_size
    q_size--;
    // reset index when queue becomes empty
    if(Empty()){
        Clear();
    }
}

template <typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}

#endif
```

[OUTPUT]

```
Front: 1 Back: 1 Size: 1
Front: 1 Back: 2 Size: 2
Front: 1 Back: 3 Size: 3
Front: 1 Back: 4 Size: 4
Front: 1 Back: 5 Size: 5
Front: 1 Back: 6 Size: 6
The Queue is Full.
Front: 2 Back: 6 Size: 5
Front: The Queue is Empty.
0 Back: The Queue is Empty.
0 Size: 0
```

[ANALYSIS]

Now in the array implementation It is the same approach as the linked list but in an array form. They both have a class structure that sets up the basic layout for A **FIXED-SIZE QUEUE** compared to a linked list that doesn't have a fixed size storage. It uses template parameters. Inside the **private block**, it **declares variables** that **track size, total, capacity, and front and back indexes**. It also has a pointer called **q_array** which will be used

to allocate the array dynamically. Implementation as shown:

```
private:
    //pointer to dynamically allocated array
    T*  q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front, q_back;
```

Additionally, here in array consists the class constructor. it initializes the queue array when a new object is created. The constructor **takes a capacity limit**, sets the **starting size to 0,** sets the **front index to 0,** and the **back index at -1.** As shown below:

```
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}
```

The **empty() and the Size() function** is the line of codes that serves as a monitoring tool and an inspector that checks if the queue contains any active data values **(isEmpty())**, and reports the exact amount of items currently living inside the queue **(size)**. The empty() function returns a boolean value of true if the container is completely vacant, or false if there are active data elements waiting to be processed.

The **enqueue() function** is the one that adds a new item to the back of the queue using **circular array logic**. It first checks if the current size equals total capacity, printing a full message and stopping if no space remains. Using the **modulo operation** it **calculates** the **new back index**, **inserts the new value into that slot,** and **increments the size counter by one**. As shown below:

```
q_back = (q_back + 1) % q_capacity;
// assign the q_array with the updated q_back to
q_array[q_back] = value;
//increment the q_size
q_size++;
```

Same with the **dequeue() function**, but it is the one that removes at the front of the queue. It uses a **modulo operator** to advance the front index tracker to the next available slot and drops the size counter by one (**decrement**). Once the queue is completely empty, it **calls clear() function** to reset the indexing tracker back to their starting position. As shown below:

```
    //compute for the q_front for circular (use the mod
    q_front = (q_front + 1) % q_capacity;
    //decrement the q_size
    q_size--;
    // reset index when queue becomes empty
    if(Empty()){
        Clear();
    }
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

[SOURCE CODE]

```
#include <iostream>
#include <queue>
#include "printerQueue.h"
#include <iomanip>
#include <chrono>
#include <thread>

int main() {
    Printer officePrinter;


    std::cout << "\n\t    >>> Shared Office Printer <<<\n\n";

    // Multiple users sending print requests
    std::cout << "=======================================================\n";
    std::cout << "      >>> Queuing Multiple Print Requests... <<<\n";
    std::cout << "=======================================================\n\n";
    officePrinter.queueDelay();
    officePrinter.addJob(101, "Striker", 5);
    officePrinter.queueDelay();
    officePrinter.addJob(102, "James", 12);
    officePrinter.queueDelay();
    officePrinter.addJob(103, "Jobert", 3);
    officePrinter.queueDelay();

    // Printer handles and processes all the incoming requests in FIFO order
    officePrinter.processJobs();
    officePrinter.timeDelay();
```

```
29
30      // Second wave of incoming print requests
31      std::cout << "=========================================================\n";
32      std::cout << "      >>> Queuing Multiple Print Requests... <<<\n";
33      std::cout << "=========================================================\n\n";
34      officePrinter.queueDelay();
35      officePrinter.addJob(104, "James", 25);
36      officePrinter.queueDelay();
37      officePrinter.addJob(105, "Ely", 7);
38      officePrinter.queueDelay();
39
40      officePrinter.processJobs();
41      officePrinter.timeDelay();
42
43      officePrinter.processJobs();
44
45      return 0;
46  }
```

[HEADER FILE]

```cpp
#ifndef PRINTERQUEUE_H
#define PRINTERQUEUE_H
#include <iostream>
#include <string>
#include <iomanip>
#include <thread>
#include <chrono>

// Create the Job class
class Job {
public:
    int id;
    std::string username;
    int pages;
    Job* next; // Pointer for the linked list node representation

    Job(int jobId, std::string name, int numPages) {
        id = jobId;
        username = name;
        pages = numPages;
        next = nullptr;
    }
};

// Create the Printer class using a Queue (First-Come, First-Served)
class Printer {
private:
    Job* front; // Points to the first job to be processed
    Job* rear;  // Points to the last job added

public:
    Printer() {
        front = nullptr;
        rear = nullptr;
    }
```

```
    // Add a new job to the pending print queue (Enqueue)
    void addJob(int id, std::string username, int pages) {
        Job* newJob = new Job(id, username, pages);

        if (rear == nullptr) { // If queue is empty
            front = rear = newJob;
        } else {
            rear->next = newJob;
            rear = newJob;
        }
        std::cout << "[ADDED] Job #" << id << " by " << username << " (" << pages << " pages) added to the queue.\n";
    }

    // Process all jobs currently in the queue one by one (Dequeue)
    void processJobs() {
        if (front == nullptr) {
            std::cout << "[STATUS] No pending print jobs.\n\n";
            return;
        }
        std::cout << "\n======================================================";
        std::cout << "\n >>> [STATUS] Processing Multiple Print Request... <<<\n";
        std::cout << "======================================================\n\n";
        while (front != nullptr) {
            Job* temp = front;

            // Print status
            std::cout << "[PRINTING] Processing Job #" << temp->id
                      << " for User: " << temp->username
                      << " [" << temp->pages << " pages]...\n";
            queueDelay();
            // Move front pointer forward
            front = front->next;

            // If the queue became empty, update rear pointer too
            if (front == nullptr) {
                rear = nullptr;
            }

            delete temp; // Free memory allocated for the completed job
        }
        std::cout << "\n[--- All pending print jobs completed successfully. ---]\n\n";
    }
```

```
80      // Destructor to clean up memory if program exits
81      ~Printer() {
82          while (front != nullptr) {
83              Job* temp = front;
84              front = front->next;
85              delete temp;
86          }
87      }
88
89      void timeDelay() {
90          return std::this_thread::sleep_for(std::chrono::seconds(5));
91      }
92
93      void queueDelay() {
94          return std::this_thread::sleep_for(std::chrono::seconds(1));
95      }
96  };
97
98
99
100
101 #endif
```

[OUTPUT]

```
            >>> Shared Office Printer <<<

=======================================================
      >>> Queuing Multiple Print Requests... <<<
=======================================================

[ADDED] Job #101 by Striker (5 pages) added to the queue.
[ADDED] Job #102 by James (12 pages) added to the queue.
[ADDED] Job #103 by Jobert (3 pages) added to the queue.

=======================================================
 >>> [STATUS] Processing Multiple Print Request... <<<
=======================================================

[PRINTING] Processing Job #101 for User: Striker [5 pages]...
[PRINTING] Processing Job #102 for User: James [12 pages]...
[PRINTING] Processing Job #103 for User: Jobert [3 pages]...

[--- All pending print jobs completed successfully. ---]

=======================================================
      >>> Queuing Multiple Print Requests... <<<
=======================================================

[ADDED] Job #104 by James (25 pages) added to the queue.
[ADDED] Job #105 by Ely (7 pages) added to the queue.

=======================================================
 >>> [STATUS] Processing Multiple Print Request... <<<
=======================================================

[PRINTING] Processing Job #104 for User: James [25 pages]...
[PRINTING] Processing Job #105 for User: Ely [7 pages]...

[--- All pending print jobs completed successfully. ---]

[STATUS] No pending print jobs.
```

[ANALYSIS]

The supplementary has tasked us to create a program, where we need to choose whether we will be creating the program through linked list or array implementation. Which I chose linked list implementation because I am more comfortable and knowledgeable in terms of my knowledge in linked list approach. The supplementary has tasked us with 5 objectives in creating the program thus, let's proceed on how I implemented each of one.

1. ***Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).***

In this approach, In the head file, I have created a class Job that compromises the ID for the job, the name of the user who submitted it, and the number of pages. Indicated below:

```
class Job {
public:
    int id;
    std::string username;
    int pages;
    Job* next; // Pointer for the linked list node representation

    Job(int jobId, std::string name, int numPages) {
        id = jobId;
        username = name;
        pages = numPages;
        next = nullptr;
    }
};
```

The **parameter takes the data** to be equated to the given variables and are passed through to different functions to **process each** of the **details** into the **enqueuing** and **dequeuing function**. The next pointer indicates only that it is for the node representation.

2. ***Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.***

After implementing the class Job, next it lays is the **class Printer**. Which entirely **holds** the **constructors** which sets up the tracks by assigning both the **front and rear node pointers directly to nullptr**. This sets the printer class to a **starting state** marking the **internal linked list queue as empty** and ready to accept its first Job submission details. As shown below:

```
class Printer {
private:
    Job* front; // Points to the first job to be processed
    Job* rear;  // Points to the last job added

public:
    Printer() {
        front = nullptr;
        rear = nullptr;
    }
```

Not only does the printer class have the starting point, it also includes all the functions to be called so that the Job details that will be provided will be processed given to the **parameter condition state**. Functions such as **addJob()** which is entirely another term for **enqueuing**, and processJobs() another term for dequeuing. But in terms of the problem, **addJob() function** means it is queuing up the document of the user, in a **FIFO order** and the **processJobs()** is the printing of the job given by the users. Indicated as:

```
void addJob(int id, std::string username, int pages) {
    Job* newJob = new Job(id, username, pages);

    if (rear == nullptr) { // If queue is empty
        front = rear = newJob;
    } else {
        rear->next = newJob;
        rear = newJob;
    }
    std::cout << "[ADDED] Job #" << id << " by " << username << " (" << pa
}
```

```
// Process all jobs currently in the queue one by one (Dequeue)
void processJobs() {
    if (front == nullptr) {
        std::cout << "[STATUS] No pending print jobs.\n\n";
        return;
    }
    std::cout << "\n=====================================================";
    std::cout << "\n >>> [STATUS] Processing Multiple Print Request... <<<\n";
    std::cout << "=====================================================\n\n";
    while (front != nullptr) {
        Job* temp = front;

        // Print status
        std::cout << "[PRINTING] Processing Job #" << temp->id
                  << " for User: " << temp->username
                  << " [" << temp->pages << " pages]...\n";
        queueDelay();
        // Move front pointer forward
        front = front->next;
```

3. ***To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.***

With this approach In order to fully accomplish this implementation, as I stated on the objective 2, the **Printer class** also **holds the constructors**, therefore, all of the submission details that is submitted, **follows a FIFO order** as it **queue it in the addJob() function** and treated as the first one to be **processed and printed in the processJobs() function.**

4. ***Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.***

Here, this is entirely focused on the output. In the main.CPP file, I created a **delay function** so that it is easier to visualize and to understand the **process of submission of jobs and printing of jobs**. As you can see, in the output, **Striker is the first person to submit a job**, thus **it is also the first job to be processed / printed**. To further represent that the printer is processing them one by one, the delay function I created slowly goes through all submission requests and also **slowly processes the printing request one by one in a FIFO order**. Represented:

```
// Multiple users sending print requests
std::cout << "=====================================================\n";
std::cout << "      >>> Queuing Multiple Print Requests... <<<\n";
std::cout << "=====================================================\n\n";
officePrinter.queueDelay();
officePrinter.addJob(101, "Striker", 5);
officePrinter.queueDelay();
officePrinter.addJob(102, "James", 12);
officePrinter.queueDelay();
officePrinter.addJob(103, "Jobert", 3);
officePrinter.queueDelay();

// Printer handles and processes all the incoming requests in FIFO order
officePrinter.processJobs();
officePrinter.timeDelay();

// Second wave of incoming print requests
std::cout << "=====================================================\n";
std::cout << "      >>> Queuing Multiple Print Requests... <<<\n";
std::cout << "=====================================================\n\n";
officePrinter.queueDelay();
officePrinter.addJob(104, "James", 25);
```

```
=====================================================
      >>> Queuing Multiple Print Requests... <<<
=====================================================

[ADDED] Job #101 by Striker (5 pages) added to the queue.
[ADDED] Job #102 by James (12 pages) added to the queue.
[ADDED] Job #103 by Jobert (3 pages) added to the queue.

=====================================================
 >>> [STATUS] Processing Multiple Print Request... <<<
=====================================================

[PRINTING] Processing Job #101 for User: Striker [5 pages]...
[PRINTING] Processing Job #102 for User: James [12 pages]...
[PRINTING] Processing Job #103 for User: Jobert [3 pages]...

[--- All pending print jobs completed successfully. ---]
```

5. ***Defend your choice of internal representation: Why did you use arrays or linked list?***

As for me, as I stated one reason above, a linked list is an easier approach for me since I have more knowledge of it compared to the array implementation. But other than that, **Linked list implementation** is also a good choice as it is a **fixed static array for this printer queue simulation**. I considered what if the office has numerous job submissions to be processed for the printer to handle? Compared to the **array which has a fixed size memory**

allocation, it is kind of a hassle since once the **array has reached its capacity size**, you have to wait until a pending job is processed and removed from the printing request. If it simultaneously submits a heavy document job that **exceeds that fixed array size**, the **printer** will experience a **Queue overflow** and might crash often. And that issue is **easily bypassed by the linked list implementation** approach as **it is dynamically creating and linking single node spaces on the system heap only when the addJob function() is active**, which is the **enqueuing job**. A linked list offers an **efficient way of operating at constant time**, for **both adding and removing elements** as long as it directly points the pointers to the front and rear, to **coordinate flawlessly to the chain of the constructors** and functions such as the addjob() and processJobs().

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google's overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, explanation of concepts, code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I assure that the concept I have taken has only been a source on how I should be able to apply Linked list into the explanatory the AI provided me.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.